

Process Mining and Formal Verification of the Volvo IT Incident Management Process BPI Challenge 2013

Formal Methods

September 8, 2026

Abstract

This report analyses the BPI Challenge 2013 event logs from Volvo IT Belgium’s VINST incident and problem management system. Three discovery algorithms are compared, model quality is assessed on four conformance dimensions, and eight temporal properties are verified against every case using finite-trace linear temporal logic. Three findings emerge. Rework dominates the process: fewer than half of all recorded events are the first occurrence of their activity within a case. The documented procedure of queueing an incident before working it is violated in the large majority of cases, a deviation established by formal verification rather than by inspection. And the Alpha miner fails on this log in a way that is predicted by theory rather than caused by defect, because almost every case contains a length-one loop that the algorithm cannot represent.

Contents

1	Introduction	3
1.1	Context	3
1.2	Objectives	3
1.3	Structure	3
2	Theoretical foundations	3
2.1	Event logs	3
2.2	Petri nets	4
2.3	Conformance dimensions	4
2.4	Linear temporal logic over finite traces	4
3	The dataset	5
3.1	Origin and scale	5
3.2	The activity label is a modelling decision	5
4	Process discovery	6
4.1	Algorithms	6
4.2	Results	7

5	Performance analysis	8
5.1	Method	8
5.2	Mean and median diverge	8
5.3	Where the time goes	9
5.4	Duration and variability	10
6	Formal verification	11
6.1	Specification	11
6.2	Interpretation	11
6.3	A methodological result: strong next	12
7	Aggregate assessment	12
8	Implementation	13
8.1	Technology stack	13
8.2	Module organisation	13
8.3	The loaded log	14
8.4	Verification	15
8.5	AI-assisted analysis	15
8.6	State and testability	16
9	Conclusions	16
9.1	Findings	16
9.2	Threats to validity	16
9.3	Further work	17

1 Introduction

1.1 Context

Process mining derives models of real processes from the event logs that information systems record as a by-product of their operation. Where a conventional process model states how work is supposed to proceed, a discovered model states how it actually proceeded. The gap between the two is the object of study.

This report applies process discovery, conformance checking and formal verification to the BPI Challenge 2013 dataset: the incident and problem management logs of Volvo IT Belgium, recorded by the VINST service management system. The dataset is a standard benchmark, which makes the results here comparable with published work.

1.2 Objectives

1. Derive process models from the logs and compare three discovery algorithms on four measures of model quality.
2. Specify properties of the incident process in temporal logic and verify them against every recorded case.
3. Quantify where time is spent and identify the structural sources of delay.
4. Determine whether the process as executed matches the process as documented.

1.3 Structure

Section 2 sets out the formal apparatus. Section 3 describes the logs and the one modelling decision the dataset forces. Section 4 compares the discovery algorithms; Section 5 analyses time; Section 6 presents the formal verification. Section 7 aggregates the measures, Section 8 describes the implementation, and Section 9 states the conclusions and the limits of the analysis.

2 Theoretical foundations

2.1 Event logs

An event log is a multiset of traces. A trace is a finite sequence of events belonging to one case, ordered by timestamp. Each event carries at minimum a case identifier, an activity name and a timestamp; the logs analysed here carry further attributes recording the organisation and resource involved.

Writing Σ for the set of activities, a trace is an element of Σ^* and a log is a multiset over Σ^* . Two traces with the same activity sequence belong to the same *variant*. The number of distinct variants relative to the number of cases measures how standardised a process is.

2.2 Petri nets

A Petri net is a tuple $N = (P, T, F)$ where P and T are disjoint finite sets of places and transitions and $F \subseteq (P \times T) \cup (T \times P)$ is the flow relation. A marking assigns tokens to places; a transition is enabled when each of its input places holds a token, and firing it consumes those tokens and produces one in each output place. A labelled net maps transitions to activities, with unlabelled (silent) transitions permitted to express routing that corresponds to no recorded event.

An accepting Petri net additionally fixes an initial and a final marking. A trace is *replayable* if some firing sequence whose visible labels spell the trace leads from the initial to the final marking.

2.3 Conformance dimensions

Model quality is not a single quantity. Four dimensions are standard, and they trade off against one another, so all four must be reported together.

Fitness The extent to which the model can replay the traces in the log. A fitness of 1 means every observed case is replayable. Low fitness means the model forbids behaviour that actually occurred.

Precision The converse: the extent to which the model forbids behaviour that never occurred. Low precision indicates an over-permissive model, in the limit a “flower” model that permits any sequence whatsoever and therefore fits perfectly while explaining nothing.

Generalisation The extent to which the model is expected to accommodate future behaviour rather than being overfitted to the observed traces.

Simplicity A structural measure penalising an excess of places, transitions and arcs relative to the behaviour captured.

Maximising fitness alone is trivial and maximising precision alone is trivial; the interesting question is the balance, and it is why Section 4 reports a table rather than a winner.

2.4 Linear temporal logic over finite traces

Properties of traces are expressed in linear temporal logic. Because a case is a *finite* sequence of events, the semantics used here are those of LTL_f , linear temporal logic interpreted over finite traces.

Given atomic propositions p drawn from the activity alphabet, formulas are built as

$$\varphi ::= p \mid \neg\varphi \mid \varphi \wedge \varphi \mid \varphi \vee \varphi \mid \varphi \rightarrow \varphi \mid \mathbf{X}\varphi \mid \mathbf{F}\varphi \mid \mathbf{G}\varphi \mid \varphi \mathbf{U} \varphi.$$

For a trace $\pi = \pi_0\pi_1 \dots \pi_{n-1}$ and position $i < n$:

$$\begin{aligned}
\pi, i \models p &\iff p \text{ holds at } \pi_i \\
\pi, i \models \mathbf{X}\varphi &\iff i + 1 < n \text{ and } \pi, i + 1 \models \varphi \\
\pi, i \models \mathbf{F}\varphi &\iff \exists j. i \leq j < n \text{ and } \pi, j \models \varphi \\
\pi, i \models \mathbf{G}\varphi &\iff \forall j. i \leq j < n \implies \pi, j \models \varphi \\
\pi, i \models \varphi \mathbf{U} \psi &\iff \exists j. i \leq j < n, \pi, j \models \psi, \text{ and } \forall k. i \leq k < j \implies \pi, k \models \varphi
\end{aligned}$$

A trace satisfies φ when $\pi, 0 \models \varphi$.

The clause for $\mathbf{X}$ deserves emphasis, because it is the source of a result reported in Section 6. In LTL_f , $\mathbf{X}$ is the *strong* next operator: at the final position of a trace there is no successor, so $\mathbf{X}\varphi$ is false there for every φ . A property written with $\mathbf{X}$ therefore fails on every trace that ends in the state the property constrains, whether or not the process misbehaved. This is correct semantics, not a defect, but it is a trap for anyone transferring formulas from the infinite-trace setting.

3 The dataset

3.1 Origin and scale

The BPI Challenge 2013 data comprises three logs recorded by Volvo IT Belgium’s VINST system: one of incidents, and two of problems, divided into those open and those closed at the time of extraction.

Log	Cases	Events	Activities	From	To
closed_problems	1,487	6,660	7	2006-01-11	2012-05-31
incidents	7,554	65,533	13	2010-03-31	2012-05-22
open_problems	819	2,351	5	2006-11-07	2012-06-15

Table 1: The three logs after loading. Case and event counts reproduce the published figures exactly, which serves as a check on the loader.

The incident log is the principal object of study; the problem logs are used for comparison in Section 7. They are not variants of one process. Median case duration differs by an order of magnitude, and the problem logs use a smaller activity alphabet.

3.2 The activity label is a modelling decision

The dataset has no activity column. Each event records a **Status** and a **Sub Status**, and what counts as an activity must be decided by the analyst. Two choices are defensible:

- **Status** alone yields four activities: **Accepted**, **Queued**, **Completed**, **Unmatched**.
- **Status+Sub Status** yields thirteen, distinguishing for instance **Accepted+In Progress** from **Accepted+Wait - User**.

The choice is consequential rather than cosmetic. Under the coarse labelling the discovered models are small and the distinction between waiting on a vendor and waiting on a user disappears entirely — and with it the central performance finding of Section 5. Under the fine labelling the models are three times larger and precision falls across every algorithm, because there is more distinct behaviour to explain.

This report adopts **Status+Sub Status** as the primary labelling, and reports both in Section 4 so that the effect of the choice is visible rather than assumed. Treating this decision as a parameter, rather than fixing it silently in the loader, is a deliberate design commitment.

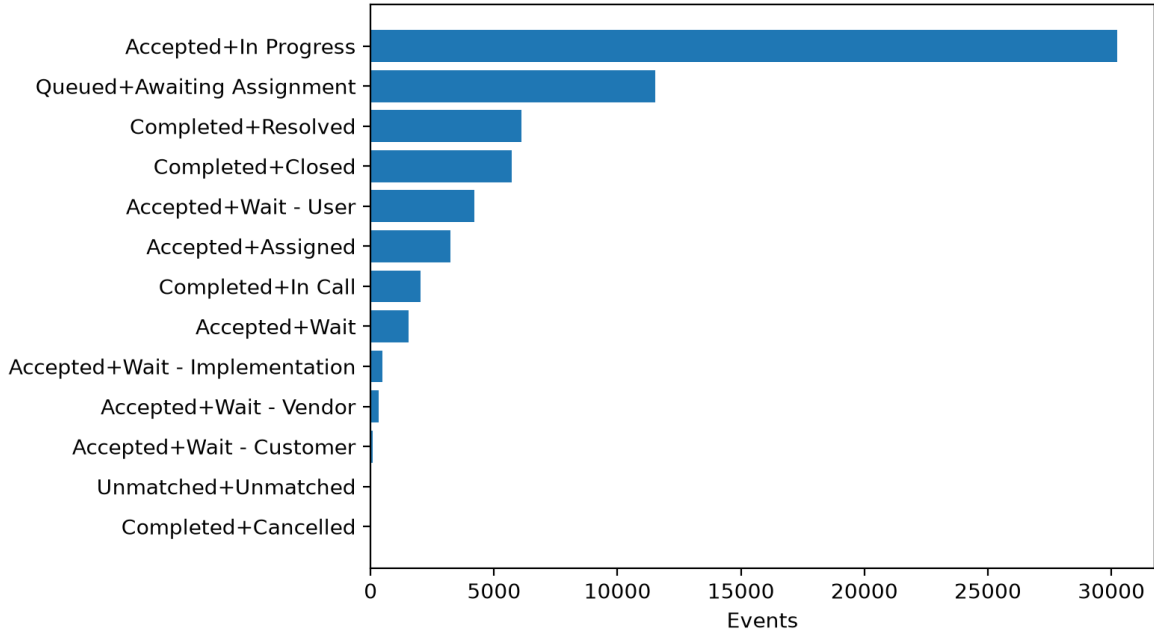


Figure 1: Event counts per activity in the incident log under the composite labelling. **Accepted+In Progress** accounts for the plurality of all events.

4 Process discovery

4.1 Algorithms

Three algorithms are compared.

Alpha miner Infers ordering relations from directly-follows pairs and constructs a net from them. Simple and historically foundational, but it assumes the log is complete and noise-free, and it cannot represent short loops.

Heuristics miner Weights the directly-follows relation by frequency and applies a dependency threshold, so infrequent behaviour is filtered out rather than modelled. Robust to noise.

Inductive miner Recursively partitions the directly-follows graph and emits a process tree, which is converted to a net. Guarantees soundness and, at a noise threshold of zero, perfect fitness.

4.2 Results

Mode	Algorithm	Fitness	Precision	Gen.	Simpl.	Places	Trans.
status	alpha	0.119	0.400	0.734	1.000	2	4
status	heuristics	0.994	0.678	0.767	0.622	8	15
status	inductive	1.000	0.440	0.748	0.667	17	23
status+substatus	alpha	0.127	0.176	0.770	0.789	2	13
status+substatus	heuristics	0.948	0.387	0.634	0.471	27	69
status+substatus	inductive	1.000	0.234	0.783	0.634	57	78

Table 2: Discovery on the full incident log; conformance computed on a sample of cases. Both activity labellings are shown.

Three observations follow.

The fitness–precision trade-off is textbook. The Inductive miner attains perfect fitness in both labellings, as its construction guarantees, and pays for it in precision: it buys replayability by being permissive, and it produces by far the largest net. The Heuristics miner sacrifices a little fitness for substantially better precision in a net a fraction of the size. If a single model must be shown to a human, it is the Heuristics model.

The Alpha miner collapses, and theory predicts it. Its fitness falls below 0.15 in both labellings and the net degenerates to two places. This is not a defect in the implementation. Ninety percent of cases in this log contain a length-one loop — an activity immediately repeated — and the Alpha miner provably cannot represent such loops, because the ordering relations it infers from a directly-follows pair (a, a) are contradictory. The result is a negative one worth reporting: it demonstrates the boundary of an algorithm’s applicability on a real log.

Finer labels cost precision. Moving from four activities to thirteen lowers precision for every algorithm and roughly triples model size, for the reason given in Section 3.2.

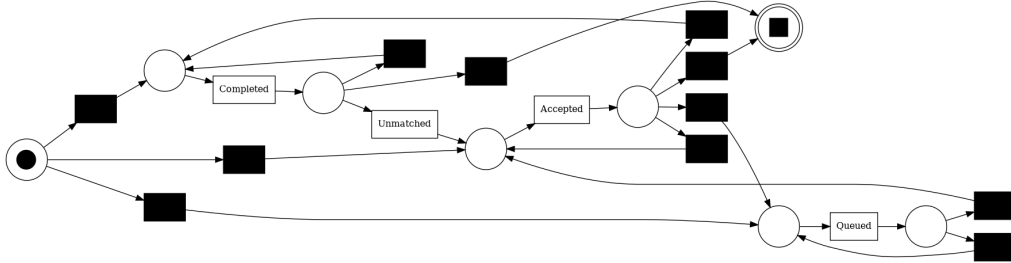


Figure 2: Heuristics miner, *Status* labelling. The best-balanced model of those compared.

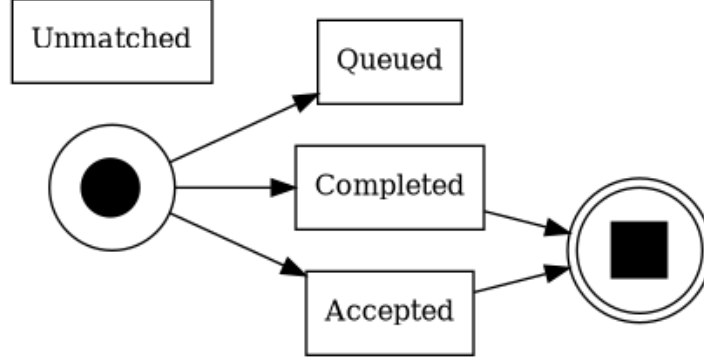


Figure 3: Alpha miner, **Status** labelling. The degenerate structure is the consequence of length-one loops, not of noise in the data.

5 Performance analysis

5.1 Method

Waiting time is measured as the interval between an event and the next event of the same case. The last event of a case has no successor and is therefore excluded, rather than being recorded as zero; conflating the two would bias every terminal activity downwards.

Activity	n	Mean (h)	Median (h)	Max (h)
Completed+Closed	143	143.31	95.28	736.50
Accepted+Wait - Vendor	313	131.63	28.15	1633.67
Accepted+Wait - Implementation	491	130.22	26.00	3500.55
Accepted+Wait - Customer	101	115.45	54.39	1488.93
Completed+Resolved	6,025	114.15	176.25	192.10
Accepted+Wait - User	4,214	109.97	26.36	5059.88
Accepted+Wait	1,533	97.99	3.45	4992.31
Completed+In Call	153	70.88	1.56	677.31
Accepted+Assigned	3,220	36.31	1.36	17334.07
Queued+Awaiting Assignment	11,544	26.16	0.59	6838.52
Accepted+In Progress	30,237	10.63	0.03	7896.28
Unmatched+Unmatched	5	0.19	0.07	0.61

Table 3: Waiting time after each activity in the incident log. Both mean and median are reported; see below for why either alone misleads.

5.2 Mean and median diverge

For **Accepted+In Progress** the median waiting time is on the order of minutes while the mean is on the order of hours. The distribution is heavy-tailed: work is normally picked up immediately and occasionally stalls for a very long time. Reporting the mean alone would suggest a slow activity; reporting the median alone would conceal the stalls. Both appear in Table 3 and in Figure 4 for this reason.

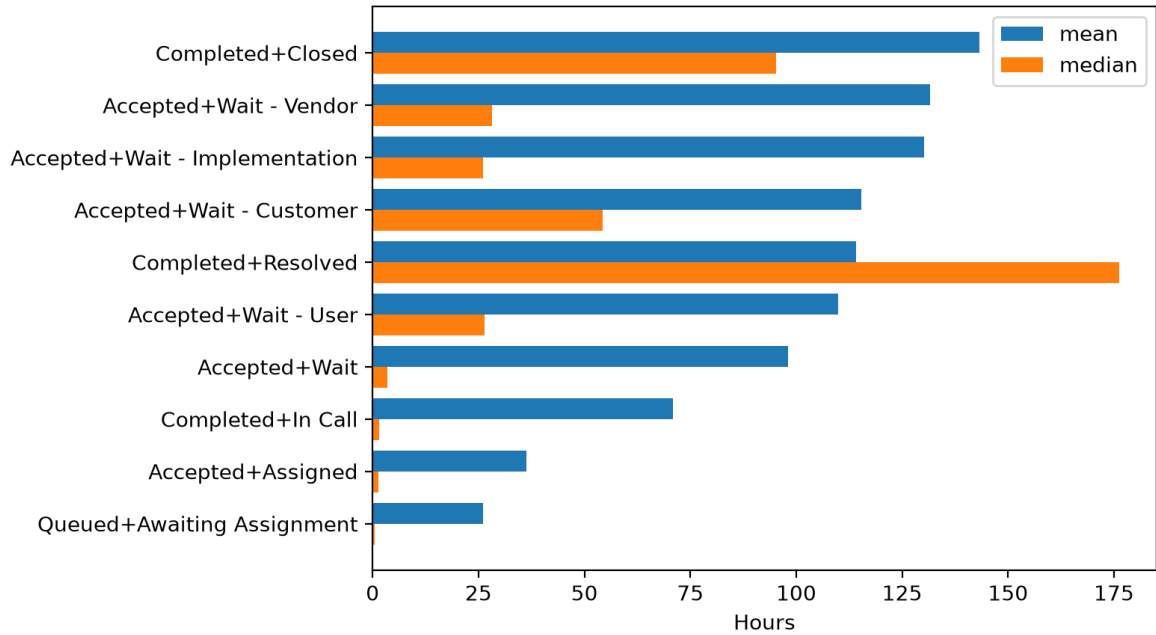


Figure 4: Mean against median waiting time for the slowest activities. Where the bars diverge sharply, the activity is usually fast and occasionally pathological.

5.3 Where the time goes

The `Wait` states dominate mean waiting time: waiting on a vendor, on an implementation, on a customer and on a user each average well over a hundred hours. This is the expected shape for a support process whose delays are external rather than internal, and it locates the improvement opportunity outside the service desk itself.

One entry requires comment. `Completed+Closed` appears in Table 3 at all, which means a substantial number of closure events were *not* the last event of their case: work continued after the incident was formally closed. Section 6 arrives at the same conclusion by a completely different route.

5.4 Duration and variability

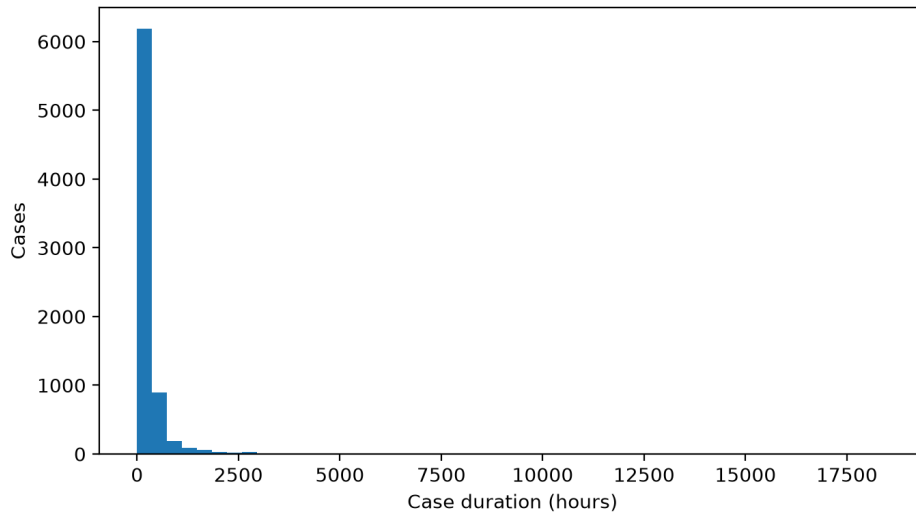


Figure 5: Distribution of case durations in the incident log.

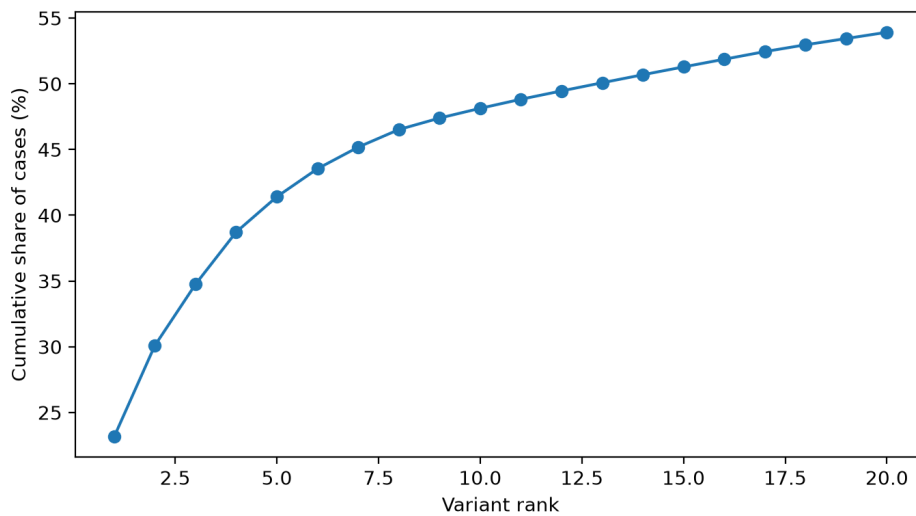


Figure 6: Cumulative share of cases covered by the most frequent variants. The curve flattens early: a large minority of cases follow paths that occur once.

The process is only weakly standardised. Distinct variants number in the thousands against seven and a half thousand cases, and the majority of variants occur exactly once. Any model of this log with high precision would be overfitted, which is the proper interpretation of the precision figures in Table 2.

6 Formal verification

6.1 Specification

Eight properties were specified in LTL_f over the composite activity alphabet, with activity labels mapped to proposition symbols (`Accepted+In Progress` becomes `accepted_in_progress`). Each property is evaluated independently against every case, and the satisfaction ratio is the share of cases satisfying it. Cases that violate a property are retained as counterexamples.

Six properties express expectations that ought to hold. Two express the documented procedure, and are included precisely because they are expected to fail; the size of the gap is the result.

Property	Satisfied	Violated	Ratio
Termination	7,546	8	0.999
Resolution	7,545	9	0.999
No work after closure	7,435	119	0.984
Queued work is picked up	7,494	60	0.992
Waiting on the user resumes	7,550	4	0.999
No unmatched events	7,549	5	0.999
Formal closure	5,591	1,963	0.740
Queueing precedes work	1,168	6,386	0.155

Table 4: Verification against all cases of the incident log.

6.2 Interpretation

The process terminates and resolves. Termination, resolution, and recovery from waiting on the user each hold for essentially every case. Whatever else is wrong with this process, cases do not disappear into it.

Work continues after closure. The property

$$G(\text{completed_closed} \rightarrow \neg F \text{accepted_in_progress})$$

fails for a small but non-trivial number of cases: an incident is formally closed and then worked again. This corroborates, from the verification side, the appearance of `Completed+Closed` in the waiting-time table of Section 5. Two independent methods agreeing is stronger evidence than either alone.

Formal closure is not universal. A quarter of incidents that are worked never reach `Completed+Closed`; they are settled as `Completed+In Call` instead. Whether this is a process defect or an undocumented but legitimate resolution path is a question for the process owner. The analysis establishes that the two routes coexist at scale.

Queueing does not precede work. The documented procedure is that an incident is queued for assignment and then taken up. Formally,

$$(\neg \text{accepted_in_progress}) \text{ U } \text{queued_awaiting_assignment}.$$

This holds for a small minority of cases. In the large majority, work begins without the incident ever having been queued. This is the “push to front” pattern for which the BPI 2013 dataset is known, and it is here established as a formal property of the log rather than inferred from inspection of variants.

6.3 A methodological result: strong next

The no-work-after-closure property was first written in the form

$$\mathbf{G}(\text{completed_closed} \rightarrow \mathbf{X} \mathbf{G} \neg \text{accepted_in_progress}),$$

which is the natural formulation in infinite-trace LTL and reports a satisfaction ratio of roughly one quarter — an apparently severe process failure. The figure is an artefact. As established in Section 2.4, $\mathbf{X}$ is false at the final position of a finite trace, and the majority of cases in this log *end* with **Completed+Closed**. Every such case falsifies the implication for reasons that have nothing to do with the process.

The **F**-based formulation given above avoids the next operator entirely and reports the true figure of roughly 98%. The two numbers differ by a factor of nearly four, and only one of them is about Volvo IT.

The general lesson is that finite-trace semantics must be applied deliberately when specifying properties of event logs, since a log is by construction a collection of finite traces. Verification tooling that silently accepts an infinite-trace formula will return a number, and the number will be wrong in a direction that looks like a finding.

7 Aggregate assessment

To compare the three logs on one scale, a composite score aggregates four normalised components: the share of cases reaching a completed state; the share of events that are the first occurrence of their activity within their case; one minus the variant ratio; and a throughput term derived from median case duration against a fixed horizon.

Log	Score	Grade	Completion	Rework	Standardisation	Throughput
closed_problems	65.9	D	1.000	0.655	0.780	0.000
incidents	74.4	C	0.999	0.481	0.698	0.748
open_problems	70.0	D	0.435	0.746	0.778	0.942

Table 5: Composite score and its components across the three logs. Higher is better on every component.

The weighting and the throughput horizon are stipulated, not derived. The score is therefore a comparison aid across logs and labellings and carries no absolute meaning; this limitation is restated in Section 9.2.

Read component-wise, one term dominates the incident log’s result. Completion is near perfect and variability is moderate, but fewer than half of all events are first occurrences of their activity. **Over half of all recorded work in the incident process is repetition.** This is the single largest lever available and it is consistent with the picture from every other section: heavy looping, work resumed after closure, and thousands of one-off variants.

8 Implementation

8.1 Technology stack

The analysis is a single self-contained Python package. Dependencies are declared in `pyproject.toml` and resolved with `uv`; the versions in Table 6 are those the reported results were produced with.

Component	Version	Role
Python	3.12	Language runtime
uv	0.11	Dependency resolution and virtual environment
pm4py	2.7.23	XES parsing, the three discovery algorithms, conformance metrics, Petri net rendering
floot	0.3.0	LTL _f parsing and evaluation over finite traces
Graphviz	2.43	External renderer invoked by pm4py for the Petri net figures
pandas	3.0	The event table underlying every analysis
Gradio	6.26	Browser dashboard
Plotly	7.0	Interactive figures in the dashboard
Matplotlib	3.11	Static figures for this report
pytest	9.1	Test suite

Table 6: Technology stack.

Two dependencies are optional extras rather than requirements. The `ai` extra carries the language model clients; the software runs, and its tests pass, without it installed, because those imports occur inside the functions that use them.

8.2 Module organisation

```
volvo/  
  config.py paths and API keys read from the environment; no domain knowledge  
  domain.py log schemas, activity-label derivation, the VINST alphabet  
  main.py command-line entry point  
  logs/  
    loader.py EventLogBundle, schema detection, normalisation  
    preprocessing.py filters and relabelling; each returns a new bundle  
  mining/  
    discovery.py discover(): Alpha, Heuristics, Inductive -> DiscoveryResult  
    conformance.py evaluate(): fitness, precision, generalisation, simplicity  
    comparison.py compare_algorithms(): one scored row per algorithm  
    performance.py waiting times, case durations, trace variants  
  verification/  
    ltl.py Alphabet, trace encoding, verify()  
    patterns.py the eight named properties and their formulas  
  analysis/  
    health.py composite score and threshold-triggered insights
```

```

    anomalies.py outlier lengths, rare activities, long waits, rare
        transitions
    predictor.py MarkovModel: first-order transition probabilities
ai/
    providers.py provider protocol, availability probing, selection order
    assistant.py fact block assembled from the loaded log, and the prompt
ui/
    app.py Gradio layout and event wiring only
    handlers.py every callback, as plain functions with no Gradio import
    charts.py Plotly figures
reporting/
    report.py collect facts, render Markdown, optional narrative section
tests/ 202 tests over the above
scripts/
    fetch_data.py resolves the three logs through the figshare API
    run_baseline.py algorithm comparison, once per labelling mode
    run_verification.py the property library against the incident log
    run_analytics.py performance, anomalies and score for every log
    run_report.py one generated report per log
    build_figures.py docs/figures/*.png
    build_tables.py docs/tables/*.tex
    build_all.py runs the above in dependency order, then both documents

```

The dependency direction is strictly downward: `ui` and `reporting` and `scripts` depend on the analysis packages, the analysis packages depend on `logs` and `domain`, and nothing depends on `ui`.

8.3 The loaded log

`logs.loader.load` dispatches on the file extension, detects the schema by matching the declared column triples in `domain.py` against the columns actually present, and returns an `EventLogBundle`: the `pm4py EventLog`, the full source frame, the schema, the labelling mode and the source filename.

Normalisation *adds* the four XES-standard columns (`case:concept:name`, `concept:name`, `time:timestamp` and, where declared, `org:resource`) to a copy of the raw frame. No column is dropped. Reducing each event to the three standard columns, as is common, would discard the support-line, ownership and country attributes that Section 9.2 identifies as the basis of any continuation of this work; these attributes have no XES-standard name and would be unrecoverable without reparsing.

The activity label is computed here, by `domain.derive_activity`, from the `Status` and `Sub Status` pair under one of three modes (`status`, `substatus`, `status_substatus`). The mode is a parameter of `load` and is then frozen into the bundle. This is the single point at which the modelling decision of Section 3.2 enters the system, which is what allows the same log to be loaded twice under different labellings and the two discovered models compared directly, as Table 2 does.

Preprocessing steps are functions from bundle to bundle. A step that would leave no events raises rather than returning an empty log.

8.4 Verification

Activity labels such as `Accepted+In Progress` are not valid proposition symbols. `verification.ltl.A` therefore builds a bidirectional map between the labels of a given log and identifiers derived from them (`Accepted+In Progress` becomes `accepted_in_progress`), resolving any collision with a numeric suffix.

Each case is one finite word. `encode_trace` converts a case’s activity sequence into one propositional interpretation per position, with every symbol of the alphabet assigned a truth value; `float` requires a total assignment at each position rather than the set of true symbols. A formula parsed by `LTLfParser` is then evaluated at position zero of each encoded trace, and the case is counted as satisfying or violating. Violating cases are retained as counterexamples up to a bound, which is where the traces quoted in Section 6 come from.

Symbols named in a formula that the current log does not contain are padded to false at every position, so a property may be evaluated against a log with a smaller alphabet. Strict parsing additionally rejects a symbol that names no activity anywhere in the dataset, which distinguishes an activity that is genuinely absent from a mistyped one.

The eight properties are declared as data — key, name, description, formula and the ratio measured on the incident log — and are evaluated by the same function that evaluates a formula typed into the dashboard. There is no separate code path behind a named property. The recorded ratios are asserted by the test suite, so a change in the analysis that moved any published figure would fail the build rather than silently update the report.

8.5 AI-assisted analysis

The dashboard can pass a block of measured facts to a language model and ask it to answer questions about them or to write a narrative section for a generated report. The block is assembled by `ai.assistant.context`: counts, the composite score and its components, duration quantiles, variant statistics, the slowest activities and the verification verdicts. The prompt instructs the model to use only those facts and to quote figures exactly. This is the only component whose output is not reproducible, and no figure in this report originates from it.

Three providers are supported behind a single protocol: a local Ollama server and the two hosted APIs. Availability is probed rather than assumed — a provider is offered only if both its client library and its key or server are present, and the local provider is preferred, so automatic selection never incurs a charge. With nothing configured the provider list is empty, the dashboard’s provider selector reads `none`, and the report generator omits the narrative section; every other feature is unaffected.

One implementation constraint is worth recording, because violating it fails silently. `config.py` reads the API keys from the environment into module constants at import time, and `providers.py` binds those constants in turn. The command-line flags that override the keys therefore have to be written into the environment before either module is imported, which is why `main.py` defers its import of the dashboard into the body of `main()`. Hoisting that import to the top of the file disables every key flag without raising anything, and the test suite still passes, because the flag handling is tested directly rather than through the import order.

8.6 State and testability

Every function in the mining, verification and analysis packages is stateless: it takes a loaded bundle and returns a value, holding no module-level state and caching nothing. The only mutable state in the running dashboard is the loaded bundle itself, held per browser session; two sessions are independent, and nothing is shared server-side. Results are recomputed on each request rather than cached.

That is what makes the analysis testable without a web server. The dashboard callbacks are ordinary functions in `ui/handlers.py` with no Gradio import, so they are called directly by the tests; the suite comprises 202 tests over the package, including regression tests that pin the measured figures reported here against the real logs.

All tables and figures in this report are generated from the analysis code by `scripts/build_tables.py` and `scripts/build_figures.py`. No measured value has been transcribed by hand.

9 Conclusions

9.1 Findings

1. **Rework dominates.** Fewer than half of the events in the incident log are the first occurrence of their activity within the case. The majority of recorded work is repetition, and this is the principal driver of the aggregate score.
2. **The documented procedure is not the executed procedure.** Queueing before work holds for a small minority of cases; the “push to front” pattern is dominant. This was established by formal verification over every case, not by sampling or inspection.
3. **The Alpha miner is inapplicable to this log.** Its failure is predicted by the prevalence of length-one loops and constitutes a demonstration of a known theoretical boundary rather than an empirical surprise.

9.2 Threats to validity

- Conformance metrics are computed on a sample of cases rather than the full log, because alignment-based computation does not terminate in reasonable time at full scale. Sampling is deterministic but not random.
- Token-based replay, used for the headline figures, is an approximation and is known to be optimistic on unsound nets. Alignment-based computation is implemented and available, and agrees on the sample.
- The composite score’s weights and horizon are stipulated. Component values are meaningful; the aggregate is meaningful only comparatively.
- `Unmatched+Unmatched` is an artefact of the VINST export rather than a process step. It is retained rather than filtered, so that its prevalence is visible, but it should not be read as work.
- Findings concern what the system recorded. Work performed but not recorded is invisible to every method used here, and a process with poor recording discipline will appear to have more rework than it has.

9.3 Further work

The loader retains the organisational attributes that this analysis does not yet use. The natural continuations are handover-of-work analysis across the support lines, quantification of the push-to-front pattern by organisation rather than by trace shape, and comparison of the incident and problem processes as distinct but interacting systems. Each is supported by the data already loaded.